

BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION Artificial Intelligence

DIPLOMA THESIS

Real-Time Intrusion Detection for Web Applications Using Deep Learning

Supervisor
Alexandra-Ioana Albu

Author
Rares Marta

2026

ABSTRACT

This thesis aims to design and implement a real-time intrusion detection system capable of analyzing web traffic and identifying malicious behavior using deep learning methods. The work is situated at the intersection of artificial intelligence and cybersecurity and focuses on building a practical system that can operate in a controlled web environment.

The thesis will include a study of the relevant literature in intrusion detection, machine learning, and deep learning for cybersecurity, with particular attention to approaches designed for traffic classification and malicious behavior detection. It will also analyze the challenges involved in evaluating such systems, especially with respect to realism, model generalization, and real-time deployment constraints.

The practical component of the thesis will consist of data processing, model training and evaluation, and the implementation of a demonstration environment in which traffic generated in a controlled web setting is monitored and classified in real time. The originality of the project is expected to come from combining deep learning-based intrusion detection with an interactive demonstration platform that highlights real-time monitoring and alert generation.

Contents

1	Introduction	1
2	Theoretical Background	3
2.1	Intrusion Detection Systems	3
2.1.1	Signature-Based Detection	3
2.1.2	Anomaly-Based Detection	4
2.1.3	Key Challenges	5
2.2	Deep Learning for Network Intrusion Detection	5
2.2.1	Multi-Layer Perceptrons	6
2.2.2	Other Deep Learning Architectures for IDS	7
2.3	The CICIOT2023 Dataset	8
2.3.1	Testbed and Data Collection	9
2.3.2	Attack Taxonomy	9
2.3.3	Feature Set	9
2.3.4	Dataset Statistics and Class Distribution	11
2.3.5	Comparison with Earlier Datasets	11
2.3.6	Known Limitations	12
3	Application Requirements and Specification	13
3.1	Project Goal and Scope	13
3.2	Stakeholders and Actors	14
3.3	Functional Requirements	14
3.3.1	Pipeline Requirements	15
3.3.2	Demo Requirements	16
3.4	Non-Functional Requirements	17
3.5	Use Cases	18
3.5.1	UC-1: Train and Persist a Model Variant	19
3.5.2	UC-2: Classify a Captured Packet Trace via the Web Demo (F1)	19
3.5.3	UC-3: Classify a Pre-Extracted CSV via the Web Demo	20
3.5.4	UC-4: Switch Model Variant Mid-Session	20
3.5.5	UC-5: Expose the Demo via a Public Tunnel	21

3.6	System Specification	21
3.6.1	High-Level Architecture	21
3.6.2	Technology Stack and Rationale	21
3.6.3	Constraints and Assumptions	23
4	Application Design, Implementation and Testing	25
4.1	Architectural Overview	25
4.1.1	Component Decomposition	25
4.1.2	Artefact Contract	26
4.2	Data Pipeline Design	27
4.2.1	Ingestion and Deduplication	27
4.2.2	Per-Class Subsampling	27
4.2.3	Caching	27
4.2.4	Cleaning	28
4.2.5	Three Split Strategies	28
4.2.6	Train-Only Preprocessing Fit	28
4.3	Model Architecture and Training	29
4.3.1	MLP Topology	29
4.3.2	Class-Weighted Cross-Entropy	29
4.3.3	Optimiser, Schedule, and Early Stopping	29
4.4	Inference Component Design	30
4.4.1	Preprocessing Mirror	30
4.4.2	Confidence and Label Decoding	30
4.5	Capture-to-CSV Bridge Design	30
4.5.1	Backend Discovery	31
4.5.2	Subprocess Hygiene	31
4.6	Web Application Design	31
4.6.1	Filesystem-Driven Variant Discovery	31
4.6.2	Two Symmetric Upload Paths	31
4.6.3	Localhost Binding by Default	32
4.7	Implementation Notes	32
4.7.1	Memory and I/O Discipline	32
4.7.2	Determinism	32
4.7.3	Logging and Observability	33
4.7.4	Configuration Surface	33
4.7.5	Dependency Footprint	33
4.8	Testing Strategy	33
4.8.1	Pipeline-Level Sanity Checks	34
4.8.2	Artefact Round-Trip Test	34

4.8.3	Latency Benchmark	34
4.8.4	F1 Functional Test — Capture to Verdict	34
4.8.5	Negative-Path Testing	35
4.9	F1 User Manual	35
4.9.1	Step 1 — Install Dependencies	35
4.9.2	Step 2 — Provide a CICFlowMeter Backend	36
4.9.3	Step 3 — Launch the Demo	36
4.9.4	Step 4 — Choose a Model Variant	36
4.9.5	Step 5 — Submit a Capture	36
4.9.6	Step 6 — Read the Output	37
4.9.7	Step 7 — (Optional) Expose the Demo Publicly	37
4.9.8	Step 8 — Capture Screenshots for the Defense Package	37
5	Results and Discussion	39
6	Conclusions and Future Work	40
	Bibliography	41

Chapter 1

Introduction

Intrusion Detection Systems (IDS) are a core defensive layer in modern networks, monitoring traffic to detect malicious behaviors such as denial-of-service attacks, reconnaissance scans, brute-force attempts, and data exfiltration. As the number of connected devices continues to grow — particularly in the Internet of Things (IoT) — the volume and diversity of network traffic have increased dramatically, exposing the limitations of traditional signature-based approaches that rely on manually maintained rule sets.

Deep learning offers a promising alternative by learning complex, non-linear representations directly from network traffic data, enabling models to detect both known and previously unseen attack patterns. However, a gap remains between the strong benchmark results reported in the research literature and the practical deployment of such models in real-time environments.

This thesis addresses that gap by designing, implementing, and evaluating a deep learning-based intrusion detection system capable of operating in real time. The work focuses on the CICIoT2023 dataset, a large-scale IoT intrusion detection benchmark containing over 46 million labeled network flow records across 33 attack types. A Multi-Layer Perceptron (MLP) is developed as the primary supervised classifier, evaluated against tree-based baselines (Random Forest, XGBoost) on the same train/validation/test splits to verify that the deep model is the best choice for this tabular task. Beyond offline evaluation, the thesis includes a demonstration environment in which scripted attacks are launched against a controlled web application, and the trained model classifies the resulting traffic flows in near real time.

The remainder of this thesis is organized as follows:

- **Chapter 2** presents the theoretical background, covering intrusion detection systems, deep learning for traffic classification, and the CICIoT2023 dataset.
- **Chapter 3** states the application requirements and specification, defining the problem to be solved, the actors involved, and the functional and non-functional

constraints the system must satisfy.

- **Chapter 4** describes the design, implementation, and testing of the application, including the data pipeline, model architecture, training procedure, the live-demo environment, and the verification of the core functionality.
- **Chapter 5** reports experimental results and discusses the findings.
- **Chapter 6** concludes the thesis and outlines directions for future work.

Chapter 2

Theoretical Background

This chapter presents the theoretical foundations underlying the work developed in this thesis. It begins with an overview of intrusion detection systems and their role in network security, then introduces deep learning as a tool for traffic classification, with a focus on multi-layer perceptrons. Finally, it describes the CICIoT2023 dataset, which serves as the training and evaluation basis for the models developed in subsequent chapters.

2.1 Intrusion Detection Systems

An Intrusion Detection System (IDS) is a security mechanism that monitors network traffic or host activity to identify malicious behavior. As networks have grown in scale and complexity, traditional perimeter defenses such as firewalls have proven insufficient on their own: attackers routinely bypass static rules and exploit new vulnerabilities, while modern networks exhibit high throughput, heterogeneity, and continual change [8]. IDS complement these defenses by analyzing traffic patterns and flagging suspicious activity.

IDS can be broadly categorized into two paradigms based on their detection methodology: signature-based and anomaly-based systems.

2.1.1 Signature-Based Detection

Signature-based IDS, exemplified by tools such as Snort and Suricata, operate by scanning network traffic for predefined patterns of known malicious activity [11]. These signatures can describe byte sequences, header anomalies, or protocol violations. Signature-based systems are highly effective at detecting known attacks with high precision, and they produce few false positives when the signature database is well-maintained.

However, their static nature makes them fundamentally limited against novel threats. Zero-day exploits, polymorphic attacks, and previously unseen attack variants will not match any existing signature and will therefore pass undetected. Additionally, signature databases require continuous manual updates to remain effective, which introduces an inherent lag between the emergence of a new attack and the availability of its corresponding signature.

2.1.2 Anomaly-Based Detection

Anomaly-based IDS take a different approach: instead of matching against known attack patterns, they learn a model of normal network behavior and flag deviations from this baseline as suspicious. This paradigm has the significant advantage of being able to detect previously unseen attacks, since any traffic that deviates sufficiently from the learned normal profile will trigger an alert.

The trade-off is a higher false positive rate. Unusual but legitimate traffic — such as a sudden spike in usage during a product launch, or an uncommon protocol used by a newly deployed application — can be incorrectly flagged as malicious. The sensitivity of anomaly-based systems depends heavily on the quality of the normal behavior model and on the threshold chosen to separate normal from anomalous activity.

Figure 2.1 illustrates the typical architecture of a real-time deep learning-based IDS, showing the end-to-end pipeline from raw packet capture to alert generation.

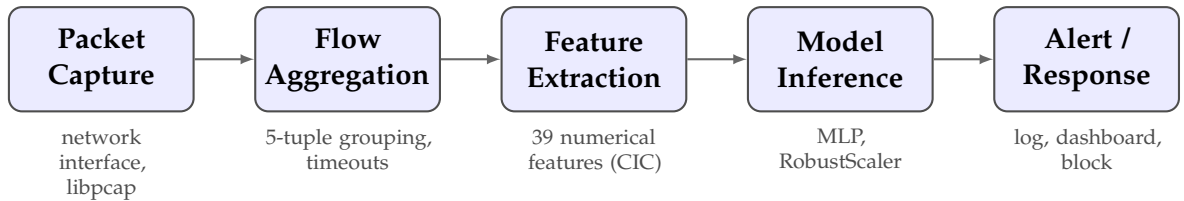


Figure 2.1: End-to-end architecture of a real-time deep learning-based intrusion detection system.

From a machine learning perspective, intrusion detection maps naturally to classification. In its simplest form, the task is binary: distinguish benign traffic from malicious flows. In practice, multi-class setups are common, with systems trained to recognize distinct categories of attacks such as DDoS, denial-of-service, reconnaissance, or botnet activity. The input is typically tabular flow data, where each flow is described by numerical features such as packet counts, byte rates, and TCP flag statistics [12].

2.1.3 Key Challenges

Deep learning models for IDS must operate under several challenges that frequently undermine the transfer from benchmark performance to real-world deployment:

- **Class imbalance:** In most network environments, benign traffic vastly outnumbers attack traffic. Benchmark datasets reflect this — for example, in CICIoT2023, DDoS attacks account for approximately 73% of all records, while web-based attacks represent less than 0.1% [10]. This imbalance can cause models to favor the majority class, making accuracy a misleading metric and requiring techniques such as weighted loss functions or stratified sampling.
- **Concept drift:** Network traffic distributions change over time due to evolving user behavior, new applications, updated protocols, and shifting attacker strategies. A model trained on a static dataset may degrade in production as the traffic it encounters diverges from its training distribution [3].
- **Dataset realism:** Most benchmark datasets are generated in controlled testbed environments rather than captured from production networks. While testbeds allow for precise labeling of attacks, they may not fully capture the diversity and complexity of real-world traffic, including encrypted communications, multi-stage attacks, and the noise inherent in large-scale networks [8].
- **Deployment constraints:** A practical IDS must meet latency and throughput requirements, integrate with existing security infrastructure, and remain maintainable over time. These operational aspects are frequently underreported in the research literature, which tends to focus on classification accuracy on static benchmarks [4].

2.2 Deep Learning for Network Intrusion Detection

Traditional machine learning approaches to IDS — such as Support Vector Machines, Decision Trees, and Random Forests — depend on manually engineered features and often struggle with high-dimensional, imbalanced, and non-stationary data. Deep learning offers an alternative by learning hierarchical representations directly from raw or lightly processed inputs. In the IDS domain, the literature reports a wide range of deep learning architectures, including deep feed-forward networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs) with Long Short-Term Memory (LSTM) units, autoencoders, and more recently transformer-based models with self-attention [8].

This section focuses on the multi-layer perceptron (MLP), which serves as the primary model in this thesis, and briefly surveys other architectures for context.

2.2.1 Multi-Layer Perceptrons

A Multi-Layer Perceptron (MLP), also referred to as a fully connected feedforward neural network or deep neural network (DNN), is the foundational deep learning architecture for supervised classification. An MLP consists of an input layer, one or more hidden layers, and an output layer, where each layer is fully connected to the next.

Formally, given an input feature vector $\mathbf{x} \in \mathbb{R}^d$ representing a network flow described by d numerical features, each hidden layer l computes:

$$\mathbf{h}_l = \sigma(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l) \quad (2.1)$$

where $\mathbf{W}_l$ and $\mathbf{b}_l$ are the learnable weight matrix and bias vector of layer l , and σ denotes a non-linear activation function, typically the Rectified Linear Unit (ReLU), defined as $\text{ReLU}(x) = \max(0, x)$. The input to the first hidden layer is the feature vector itself: $\mathbf{h}_0 = \mathbf{x}$.

For binary classification (benign vs. attack), the output layer applies a sigmoid activation:

$$\hat{y} = \sigma_{\text{sig}}(\mathbf{w}^T \mathbf{h}_L + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{h}_L + b)}} \quad (2.2)$$

where $\mathbf{h}_L$ is the output of the last hidden layer and $\hat{y} \in (0, 1)$ represents the predicted probability that the input flow is an attack. The model is trained by minimizing the binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2.3)$$

using backpropagation and an optimizer such as Adam [13].

MLPs are particularly well suited for flow-level intrusion detection for several reasons:

- **Native fit for tabular data:** Network flow features form fixed-length numerical vectors — exactly the input format MLPs are designed for. Unlike CNNs (which assume spatial structure) or RNNs (which assume sequential structure), MLPs make no structural assumptions about the input, which is appropriate when each sample is an independent flow record with heterogeneous features [6].
- **Fast inference:** MLP inference consists of a sequence of matrix multiplications, which are highly optimized on modern hardware. This makes MLPs particularly suitable for real-time IDS, where low latency is critical. A small MLP can process tens of thousands of flows per second on a single CPU core [13].

- **Competitive performance on tabular tasks:** Recent research has shown that well-regularized MLPs are competitive with both tree-based models and more complex neural architectures on tabular classification tasks. Kadra et al. demonstrated that an MLP with a carefully tuned combination of regularization techniques (dropout, weight decay, batch normalization) outperformed both XGBoost and specialized architectures such as TabNet on nearly half of the 40 datasets evaluated [6].
- **Strong baseline:** In IDS research, MLPs consistently achieve 95–99% accuracy on standard benchmarks, making them an honest and competitive baseline against which to measure more complex architectures [13, 5].

Key training considerations for MLP-based IDS include:

- **Class imbalance handling:** Weighted loss functions assign higher weight to minority (attack) classes in the cross-entropy loss. Alternatively, oversampling techniques such as SMOTE can be used to generate synthetic minority samples, or focal loss can be applied to down-weight well-classified examples and focus training on hard cases.
- **Regularization:** Dropout (randomly zeroing neuron outputs during training with a probability typically between 0.2 and 0.5) prevents co-adaptation of neurons. Weight decay (L2 regularization) penalizes large weights. Early stopping monitors validation loss and halts training when it begins to increase.
- **Feature scaling:** Network flow features have vastly different scales (e.g., packet counts vs. flow durations in microseconds). Standard normalization — fitting a scaler on the training set and applying it to validation and test sets — is essential for stable gradient-based optimization.

2.2.2 Other Deep Learning Architectures for IDS

Beyond MLPs, several other deep learning architectures have been applied to intrusion detection:

- **Convolutional Neural Networks (CNNs):** CNNs exploit local correlations by applying convolutions to reshaped tabular features, treating them as one-dimensional or two-dimensional grids. Convolutions can extract local patterns that correspond to protocol-specific or timing-related signatures of attacks [14].
- **Recurrent Neural Networks (RNNs/LSTMs):** LSTM-based IDS capture temporal dependencies in sequential traffic data, which is particularly useful in

industrial or cyber-physical system settings where context over time is critical [7].

- **Autoencoders:** Autoencoders learn to reconstruct their inputs through a bottleneck latent representation. When trained only on benign traffic, the reconstruction error serves as an anomaly score: flows with unusually high error are flagged as attacks. This unsupervised approach does not require labeled attack data, but depends heavily on threshold selection and training data quality [9].
- **Transformers:** Transformers rely on self-attention to model global relationships among features. In tabular IDS adaptations, each feature can be treated as a token and embedded into a vector space. The key advantage is the ability to model global feature interactions rather than relying on fixed local receptive fields. However, transformer models are typically supervised, requiring labeled attack data, and can be compute-intensive [15].

Table 2.1 summarizes the key differences between supervised and unsupervised deep learning paradigms for IDS.

Aspect	Unsupervised / Hybrid	Supervised
Label requirement	Low–Medium	High
Zero-day sensitivity	Higher potential	Typically lower
Calibration effort	High (thresholding)	Medium
Benchmark performance	Medium–High	High
Inference speed	Fast	Varies

Table 2.1: Qualitative comparison of unsupervised/hybrid and supervised DL-IDS paradigms.

2.3 The CICIoT2023 Dataset

The CICIoT2023 dataset, published by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick, is a large-scale benchmark dataset for IoT intrusion detection research [10]. It was selected for this thesis based on several criteria: recency (2023), scale (over 46 million records), diversity of attack types (33 distinct attacks across 7 categories), and growing adoption in the research community.

2.3.1 Testbed and Data Collection

The dataset was generated in a physical laboratory environment simulating a realistic smart home network. The testbed consists of **105 real IoT devices**, including smart cameras, speakers, plugs, lights, sensors, and home automation controllers from various manufacturers. Of these, 67 devices are directly involved in attack scenarios as either attackers or victims, while 38 Zigbee/Z-Wave devices are connected through 5 hubs.

The attack infrastructure uses 7 Raspberry Pi 4 devices configured as attackers. For distributed attacks (DDoS), multiple Raspberry Pis coordinate via an SSH-based master-client configuration. Network traffic is passively captured using a Gigamon network tap and two monitoring stations running Wireshark, producing approximately 548 GB of raw PCAP data.

Feature extraction is performed using **DPKT**, a Python packet parsing library. Unlike CICFlowMeter (used in earlier CIC datasets such as CICIDS2017), DPKT operates at the packet level with a fixed-size packet window, computing features over a sequence of consecutive packets between two hosts rather than per-flow. This approach offers finer granularity and is more suitable for real-time detection, as features can be computed as soon as enough packets in the window are observed, without waiting for a flow to terminate [10].

2.3.2 Attack Taxonomy

The dataset contains **33 distinct attack types** organized into 7 categories, plus benign traffic, for a total of 34 class labels. Table 2.2 summarizes the attack categories, the number of attacks in each, and the tools used for generation.

2.3.3 Feature Set

The original CICIOT2023 paper describes a 46-feature representation produced by the DPKT-based extractor used during dataset construction. The features can be grouped as follows:

- **Flow timing** (3 features): flow duration, time-to-live (TTL), and inter-arrival time with the prior packet.
- **Rate and throughput** (3 features): overall packet transmission rate, source rate, and destination rate.
- **Header and protocol** (2 features): header length and protocol type (IP, UDP, TCP, IGMP, ICMP).

Category	Count	Examples	Tools
DDoS	13	UDP Flood, SYN Flood, SlowLoris, HTTP Flood	hping3, golang-httpflood
DoS	4	TCP Flood, UDP Flood, SYN Flood, HTTP Flood	hping3, udp-flood
Reconnaissance	5	PingSweep, PortScan, OSScan, VulnerabilityScan	nmap, fping, vulscan
Web-Based	6	SQL Injection, XSS, Command Injection, Backdoor	DVWA, Remot3d, BeEF
Brute Force	1	DictionaryBruteForce	nmap, hydra
Spoofing	2	ARP Spoofing, DNS Spoofing	ettercap
Mirai	3	GREIP Flood, GREeth Flood, UDP-Plain	Adapted Mirai source

Table 2.2: CICIOT2023 attack taxonomy: 7 categories with 33 attack types.

- **TCP flag features** (7 features): binary indicators for FIN, SYN, RST, PSH, ACK, ECE, and CWR flags.
- **Packet count features** (5 features): counts of ACK, SYN, FIN, URG, and RST packets.
- **Protocol indicator features** (14 binary features): binary indicators for HTTP, HTTPS, DNS, Telnet, SMTP, SSH, IRC, TCP, UDP, DHCP, ARP, ICMP, IPv, and LLC.
- **Packet length statistics** (6 features): total sum, minimum, maximum, average, standard deviation, and individual packet length.
- **Aggregate statistical features** (6 features): packet count in flow, magnitude, radius, covariance, variance, and weight — derived from combined incoming and outgoing packet statistics.

At the time of writing, the publicly available CSV release distributed by the Canadian Institute for Cybersecurity contains a **39-feature subset** of the 46 features described in the paper. Specifically, the public CSV omits the source and destination rate features, the URG packet count, and four of the six aggregate statistics (*Magnitude*, *Radius*, *Covariance*, and *Weight*). The 39-feature version is the basis for all experiments and the live demonstration in this thesis. This is acknowledged as a deliberate scope

decision: results in this work are not numerically directly comparable to papers that report on the full 46-feature variant, and that gap is part of the methodological story rather than an oversight.

2.3.4 Dataset Statistics and Class Distribution

The full dataset contains approximately **46.7 million records** distributed across 169 CSV files, totaling approximately 13 GB uncompressed. The class distribution is severely imbalanced:

- **Benign traffic:** approximately 1.1 million samples (2.4% of the dataset).
- **DDoS attacks:** approximately 73% of all records, with DDoS-ICMP_Flood alone accounting for over 7.2 million samples.
- **Web-based attacks:** fewer than 25,000 samples combined, with the rarest classes (XSS, Backdoor_Malware, CommandInjection) containing only around 100 samples each.

The most extreme class ratio is approximately 5,751:1 between DDoS-ICMP_Flood and the smallest attack class. This severe imbalance necessitates careful handling during model training, either through stratified sampling, class weighting, or a combination of both. Due to the dataset’s size and computational constraints, most published experiments operate on stratified subsamples of 1–5 million records, preserving the original class distribution [10].

2.3.5 Comparison with Earlier Datasets

Table 2.3 compares CICIoT2023 with other commonly used IDS benchmark datasets.

Dataset	Year	Records	Features	Attack types	Domain
NSL-KDD	2009	150K	41 (custom)	4 classes	General
UNSW-NB15	2015	2.5M	49 (Argus+Bro)	9 classes	General
CICIDS2017	2017	2.8M	78 (CICFlowMeter)	14 attacks	Enterprise
CICIoT2023	2023	46.7M	39 (DPKT, CSV)	33 attacks	IoT smart home

Table 2.3: Comparison of commonly used IDS benchmark datasets.

CICIoT2023 offers several advantages over older benchmarks: it is significantly larger, contains more diverse and modern attack types (including IoT-specific attacks such as Mirai botnet variants), and uses real IoT hardware rather than simulated traffic generators. Its primary limitations include the lack of encrypted traffic analysis, the restriction to a smart home IoT context (excluding industrial or medical IoT), and the fact that attacks are executed individually rather than as multi-stage campaigns [10].

2.3.6 Known Limitations

Several limitations of the CICIoT2023 dataset should be acknowledged:

- **Simulated environment:** Although real IoT devices are used, the attacks are executed in a controlled laboratory setting. Real-world attackers combine techniques, adapt strategies, and conceal their actions — none of which are captured in a static dataset.
- **No encrypted traffic:** The features are extracted from packet headers and metadata. While HTTPS is flagged as a binary indicator, the dataset does not contain features derived from encrypted payload inspection. IoT traffic is increasingly encrypted, which limits the generalizability of models trained on this data.
- **Smart home scope:** The testbed represents a single domain. Industrial IoT, healthcare IoT, and other verticals are not represented, and models may not transfer to those environments.
- **Severe class imbalance:** As noted above, the most extreme ratio exceeds 5,000:1, requiring careful handling during both training and evaluation.
- **Feature documentation:** The exact formulas for some derived features (Magnitude, Radius, Covariance, Variance, Weight) are not fully documented with mathematical definitions in the original paper.

Chapter 3

Application Requirements and Specification

This chapter defines the application that constitutes the practical contribution of the thesis. The overall objective is twofold: to deliver a defensible, reproducible deep-learning pipeline trained on the CICIoT2023 dataset, and to expose the resulting model through a live demonstration in which the user can submit captured network traffic and observe per-flow intrusion-detection verdicts. The chapter starts from this objective, identifies the actors and use cases, lists the functional and non-functional requirements that shape the design, and concludes with the system-level specification that constrains the implementation in Chapter 4.

3.1 Project Goal and Scope

The deliverable is a real-time intrusion detection system (RT-IDS) for IoT networks built on top of CICIoT2023, structured as two cooperating subsystems:

- **Training subsystem.** A reproducible offline pipeline that ingests the raw CICIoT2023 captures, performs deduplication and per-class subsampling, builds three different train/validation/test splits, trains a deep neural model at three classification granularities (binary, attack-family, and full multi-class), evaluates the model with the metrics expected by the IDS literature, and serialises the small set of artefacts needed for inference.
- **Serving subsystem.** A web application that loads the serialised artefacts, accepts either a pre-extracted feature CSV in CIC format or a raw packet capture (in which case a flow-extraction tool is invoked to derive the feature representation), runs the trained model, and presents per-flow verdicts and an aggregated summary in the browser.

The system is targeted at an interactive, single-request setting: the user supplies a sample of traffic, the system classifies it, and the verdict is reviewed on screen. **Line-rate gateway protection** (sub-millisecond per-packet inference at multi-gigabit throughput) is explicitly out of scope. This narrower framing of “real time” is restated in the requirements below, because it directly determines which non-functional thresholds are reasonable to enforce.

Three classification granularities are required, not optional, because the comparison between them is part of the thesis contribution and matches the way most CICIoT2023 papers report results:

- **2-class:** Benign versus Attack. This is the gate that an inline production deployment would actually use.
- **8-class:** Benign plus the seven attack families (DDoS, DoS, Mirai, Reconnaissance, Spoofing, Web, BruteForce). This is the operational view a Security Operations Centre (SOC) analyst would consume.
- **34-class:** One label per specific attack variant. This is the forensic view used in published comparisons.

The same train/validation/test rows must be reused across all three granularities (only the label-mapping changes), so the comparison between granularities is not contaminated by sampling differences.

3.2 Stakeholders and Actors

The system has a small, well-defined set of human actors. Each actor’s interactions with the system constrain the user-interface design.

The application has no concept of long-lived user sessions, accounts, or persistent storage of submitted captures. All actors interact with the system over a single short-lived session, typically lasting the duration of the demonstration.

3.3 Functional Requirements

The functional requirements describe *what* the system must do. They are split into pipeline-side requirements (the offline training subsystem) and demo-side requirements (the web application). The first demo requirement, **F1**, is the core executable functionality whose end-to-end operation must be demonstrated for the assignment.

Actor	Role	Primary interaction
Researcher / thesis author	Trains and evaluates models, regenerates artefacts.	Runs the training pipeline end-to-end, iterates on splits and granularities, inspects metrics and confusion matrices.
Demo presenter	Presents the live demonstration during the thesis defense.	Launches the web application, selects a model variant, uploads a captured packet trace, narrates the verdicts.
Defense audience	Observes the demonstration.	Reads the verdict table and summary in the browser; may request that a different model variant be selected.
Thesis committee reviewer	Verifies that the system behaves as claimed.	Reproduces a single classification run from a supplied capture and compares the output to the expected label.

Table 3.1: Actors involved in the system and their primary interactions.

3.3.1 Pipeline Requirements

- **FR-P1 — Reproducible ingestion.** The pipeline shall ingest the raw per-folder CICIoT2023 CSVs, attach the folder name as the fine-grained class label, drop exact duplicate rows (the dataset contains a substantial fraction of duplicate records), and produce a single compact, columnar intermediate file. Re-running ingestion with the same seed and the same source files shall produce a byte-identical intermediate.
- **FR-P2 — Per-class subsampling.** For each fine-grained label, the pipeline shall randomly subsample at most a configurable per-class cap of rows, using a fixed random seed. Sampling shall be uniform across each class, not the leading rows; the leading rows of each attack folder come from a single capture session and would bias the model toward that session’s idiosyncrasies.
- **FR-P3 — Three split variants.** The pipeline shall generate three train/validation/test splits over the same per-class subsampled rows: **temporal** (sort the source captures per attack folder, take the earliest 70% as train, the next 15% as validation, and the latest 15% as test); **per-capture** (no source capture appears in more than one partition, so within-session redundancy cannot leak across the split); **random row** (the conventional stratified row split, retained for parity with previously published numbers).
- **FR-P4 — Three classification granularities.** For each split, the pipeline

shall train and evaluate one model per granularity (binary, attack-family, fine-grained). The same train/validation/test row indices shall be reused across the three granularities; only the label encoding differs.

- **FR-P5 — Imbalance handling.** The pipeline shall combat the severe class imbalance via class-weighted cross-entropy loss, with weights computed from the training-set class frequencies. Synthetic minority oversampling shall not be used (synthetic flow records have no operational meaning), and weighted batch sampling shall not be used (it would be redundant with weighted cross-entropy).
- **FR-P6 — Tree-based baselines.** The pipeline shall additionally train a Random Forest and a gradient-boosted tree classifier on the same splits and report the same metrics, so that the deep model can be compared against strong tabular baselines.
- **FR-P7 — Standard evaluation suite.** For each (split, granularity) pair the pipeline shall report accuracy, macro-F1, weighted-F1, macro-precision, macro-recall, the per-class precision/recall/F1 report, and the row-normalised confusion matrix.
- **FR-P8 — Latency benchmark.** For each trained model the pipeline shall measure single-flow inference latency at batch sizes [1, 32, 256, 1024] over at least 10,000 warm runs, reporting the p50, p95, and p99 percentiles end-to-end (i.e. including feature scaling, not only the network forward pass). Mean-only latency reporting is forbidden.
- **FR-P9 — Artefact serialisation.** For each (split, granularity) pair the pipeline shall persist three artefacts: the fitted feature scaler, the fitted label encoder, and the trained model weights. The serialisation format shall allow the serving subsystem to reload the artefacts without re-running training, and the naming convention shall be uniform across model variants so that new variants can be added without code changes (see Section 3.6.1).

3.3.2 Demo Requirements

- **F1 — Per-flow classification of submitted traffic.** *This is the executable functionality required by the Lab 4 assignment.* The demo shall accept either (a) a CSV file already in the CIC feature format, or (b) a raw packet capture, and shall return a per-flow classification table together with an aggregated summary. For packet-capture input, the established CIC flow-extraction tool shall be invoked

as a subprocess to derive the feature representation; the demo shall not reimplement flow extraction in Python, because doing so would risk subtle numerical drift between live and training distributions.

- **FR-D2 — Model-variant selection.** The demo shall present a dropdown listing every (split, granularity) pair for which trained artefacts exist on disk. The user shall be able to switch model variant without restarting the application; subsequent classifications shall use the newly selected variant.
- **FR-D3 — Deterministic output schema.** For every classification request, the demo shall return: (i) a one-line summary with the total number of flows classified and a breakdown by predicted label, sorted by descending count; (ii) a table where each row contains the row index in the input, the predicted label, and the classifier's confidence (the maximum softmax probability), formatted with four decimal digits.
- **FR-D4 — Self-validating input.** The demo shall verify that every uploaded CSV contains the expected feature columns and shall return a human-readable error message naming the missing columns if it does not. Flow-extraction failures on packet captures shall surface the underlying tool's error message rather than crashing the request.
- **FR-D5 — No persistent storage of user input.** Submitted captures and CSVs shall be processed in a temporary location and removed when the request ends. The application shall not log captured payloads to disk.
- **FR-D6 — Optional public exposure.** The application shall be runnable in two modes: (a) bound to localhost for an in-room defense demonstration; (b) exposed publicly via a tunnelling service on demand. The web application itself shall not embed a tunnelling client; tunnelling is delegated to a separate command-line tool launched by the presenter.

3.4 Non-Functional Requirements

The non-functional requirements describe *how well* the system must perform.

- **NFR-1 — Latency target.** End-to-end per-flow inference latency through the demo, including CSV parsing, feature scaling, the model forward pass, and result formatting, shall be below 100 ms at p95 on a consumer laptop CPU for batches of up to 1024 flows. Packet-capture parsing time is excluded from this budget because the flow-extraction tool dominates that path; capture-to-verdict latency is reported separately.

- **NFR-2 — Headline classification performance.** The temporal-split model at the 8-class granularity shall achieve at least 0.85 weighted-F1 on the held-out test set. The temporal headline number is expected to be 2–10 percentage points lower than the random-split number; that gap is treated as the honest generalisation story, not a defect.
- **NFR-3 — Reproducibility.** Every random source — numerical libraries, deep-learning framework on CPU and on GPU, and dataset samplers — shall be seeded from a single deterministic seed. Re-running the pipeline shall produce identical splits, identical class weights, and metrics within a tolerance of ± 0.001 F1.
- **NFR-4 — Memory budget.** The pipeline shall fit within 16 GB of system RAM at every stage. This rules out keeping the full multi-tens-of-millions-of-rows CSV materialised in memory and forces the use of lazy, columnar processing with on-disk intermediates.
- **NFR-5 — Hardware portability.** The pipeline shall run on CPU-only hardware. GPU acceleration shall be used opportunistically when available, but shall never be a hard dependency.
- **NFR-6 — Cross-platform demo.** The demo application shall run on both Windows and Linux, with the only platform-specific concern being the path of the flow-extraction tool’s launcher script.
- **NFR-7 — Operational simplicity.** Launching the demo shall require a single command. No additional services (databases, message queues, dedicated model servers) shall be required.
- **NFR-8 — Honest reporting.** Numerical results in the thesis shall not cherry-pick the favourable random split; the temporal split shall be the headline result. Per-class metrics shall always be reported alongside aggregate metrics, so that majority-class dominance cannot mask poor minority-class performance.

3.5 Use Cases

The use cases below describe the principal interactions with the system. Each use case is given a short identifier, the actor that initiates it, the precondition that must hold, the main flow of events, and the postcondition that holds when the use case completes successfully. Alternative flows describe the most important error paths.

3.5.1 UC-1: Train and Persist a Model Variant

Actor: Researcher.

Precondition: The raw CICIoT2023 captures are available in the dataset directory; the runtime environment satisfies the project's dependency manifest.

Main flow:

1. The researcher opens the training pipeline and configures which split strategies and which granularities to run.
2. The pipeline ingests, deduplicates, and subsamples the data; it caches the result on first run so subsequent runs skip ingestion.
3. For each requested (split, granularity) pair, the pipeline fits the scaler on the train set only, encodes labels, computes class weights, trains the model with early stopping, and evaluates on the test set.
4. The fitted scaler, label encoder, and trained model weights are written to the artefact directory under a uniform naming convention.
5. Aggregate and per-class metrics are printed; the row-normalised confusion matrix is rendered.

Postcondition: The artefact directory contains a complete artefact triple for every requested (split, granularity) pair, ready to be loaded by the serving subsystem.

Alternative flow: If a previously cached intermediate is present, ingestion is skipped and the pipeline proceeds directly to splitting.

3.5.2 UC-2: Classify a Captured Packet Trace via the Web Demo (F1)

Actor: Demo presenter.

Precondition: At least one artefact triple is present in the artefact directory; a CI-CFlowMeter backend is reachable; the web application is running and accessible in a browser.

Main flow:

1. The presenter opens the demo URL and selects a model variant from the dropdown.
2. The presenter switches to the packet-capture upload tab and uploads a capture produced by the attack-script step.
3. The application copies the capture into a temporary location, invokes the flow-extraction tool as a subprocess, and obtains a feature CSV.

4. The application loads the CSV, applies the scaler fitted on the training set, runs the model forward pass, applies softmax, and inverse-transforms the predicted indices to label strings.
5. The application renders the summary line and the per-flow verdict table in the browser.

Postcondition: The temporary location is removed; the verdict table is visible; no traffic data is persisted on disk.

Alternative flow A: If the flow-extraction tool exits non-zero, the application returns an error message containing the captured stderr and an empty table.

Alternative flow B: If no flow-extraction backend is available, the application returns a configuration message explaining how to provide one and an empty table.

3.5.3 UC-3: Classify a Pre-Extracted CSV via the Web Demo

Actor: Demo presenter or thesis-committee reviewer.

Precondition: At least one artefact triple is present in the artefact directory; the web application is running.

Main flow:

1. The actor opens the demo URL and selects a model variant.
2. The actor switches to the CSV upload tab and uploads a CSV in the CIC feature format.
3. The application validates the column set, applies the scaler, runs the model, and renders the verdict table.

Postcondition: The verdict table is visible; the uploaded CSV is removed when the request completes.

Alternative flow: If required CIC columns are missing, the application returns a single human-readable error message naming the missing columns and an empty table.

3.5.4 UC-4: Switch Model Variant Mid-Session

Actor: Demo presenter.

Precondition: The web application is running and at least two artefact triples have been discovered at startup.

Main flow:

1. After running a classification, the presenter selects a different (split, granularity) pair from the dropdown.

2. The presenter triggers classification again on the same uploaded file.
3. The application routes the request through the predictor corresponding to the new dropdown value and renders the new verdicts.

Postcondition: The verdict table reflects the newly selected model variant; no application restart was required.

3.5.5 UC-5: Expose the Demo via a Public Tunnel

Actor: Demo presenter.

Precondition: The demo is running locally; a tunnelling client is installed.

Main flow:

1. The presenter opens a second terminal and starts a tunnel that points at the local demo URL.
2. The tunnelling service prints a temporary public HTTPS URL.
3. Remote viewers reach the demo through that URL; the request flow is otherwise identical to UC-2 / UC-3.

Postcondition: The demo is reachable from outside the local network; tearing the tunnel down revokes the URL.

3.6 System Specification

The system specification translates the requirements above into the high-level architecture, the technology choices, and the interfaces that govern how components cooperate. The detailed design and implementation are presented in Chapter 4; this section fixes the contract.

3.6.1 High-Level Architecture

The system consists of two pipelines that meet at a small set of serialised artefacts. The training pipeline runs offline and produces those artefacts. The serving pipeline is the live demo, which loads them and answers classification requests. Figure 3.1 shows the data flow.

3.6.2 Technology Stack and Rationale

The technology choices are driven by the requirements above. Table 3.2 lists each concern, the selected technology, and the reason it was chosen over the obvious alternatives.

Concern	Choice	Rationale
Tabular data wrangling	Lazy columnar engine, compressed columnar intermediates	Lazy scanning lets the multi-tens-of-millions-of-rows CSV corpus be deduplicated and subsampled without ever materialising it in memory; columnar storage with strong compression cuts disk usage and reload time.
Feature scaling	Robust scaler (median / IQR)	CIC features have heavy-tailed distributions (rate, IAT, total bytes); scaling by median and IQR is more stable than zero-mean / unit-variance scaling under such tails.
Label encoding	Integer label encoder	Compact and reversible; the inverse transform is needed at inference time to display human-readable labels.
Imbalance handling	Class-weighted cross-entropy	SMOTE rejected (synthetic flow records have no operational meaning); weighted batch sampling rejected (redundant with weighted cross-entropy).
Deep model	Two-hidden-layer MLP with batch normalisation and dropout	MLPs are the natural fit for fixed-length tabular inputs, train fast on CPU, and serve as the strong baseline expected by the IDS literature.
Tree baselines	Random Forest, gradient-boosted trees	Required to demonstrate that the deep model is not strictly worse than a tree, which is the first question committees ask about tabular tasks.
Capture flow CSV →	CICFlowMeter (the CIC team's reference tool)	Reimplementing flow aggregation in Python would risk subtle numerical drift between the live-demo features and the training distribution.
Demo UI	Lightweight Python web-UI framework with file upload, dropdowns, and dataframe widgets	Lowest-friction route to a thesis-defence demo; no JavaScript required.
Public exposure	Stateless HTTPS tunnelling client	No router configuration required; the URL expires when the presenter closes the tunnel.
Reproducibility	Single deterministic seed propagated to every random source	Required by NFR-3.

Table 3.2: Technology stack of the RT-IDS²² application and rationale for each choice.

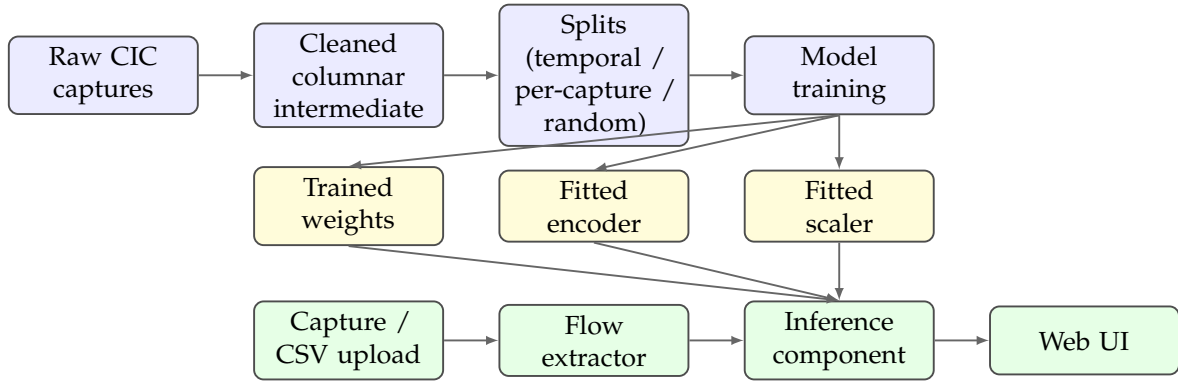


Figure 3.1: High-level architecture of the RT-IDS application. The training pipeline (blue) produces three serialised artefacts (yellow), which the serving pipeline (green) loads to answer per-flow classification requests.

3.6.3 Constraints and Assumptions

The following constraints and assumptions bound the work and are acknowledged in the methodology section of the thesis.

- **Dataset version.** The 39-feature public CSV release of CICIoT2023 is the only source of training data. The 46-feature variant used by some published comparisons was not available from the official UNB CIC distribution at the time of writing; it is mirrored on third-party platforms but its provenance and pre-processing differ from the official release. Numerical results are therefore not directly comparable across the two feature sets, and that gap is treated as a methodological scope decision rather than a defect.
- **Smart-home scope.** CICIoT2023 was captured in a smart-home testbed with 105 IoT devices. Generalisation to industrial control systems, medical IoT, or vehicular networks is not claimed.
- **No encrypted-payload features.** The dataset features are derived from headers and metadata only. Detection accuracy on heavily encrypted traffic where attack signal is concentrated in the payload is expected to be limited; web-family attacks (SQL injection, XSS, command injection) will be detected weakly because they are application-layer and the feature set is transport-layer. This is a property of the feature set, not a model defect, and is discussed openly in the results chapter.
- **Single-machine deployment.** The demo runs on the presenter’s laptop. Distributed serving, redundancy, and high-availability are out of scope.
- **Trusted local audience.** The demo does not authenticate users. The expected audience is the thesis committee in a controlled room, optionally extended via

an on-demand HTTPS tunnel for a known remote reviewer. Hardening the demo for hostile public exposure is out of scope.

- **No online learning.** The deployed model is the one trained offline by the pipeline. The demo does not adapt to feedback during the session.

Chapter 4

Application Design, Implementation and Testing

This chapter covers the design and implementation of the application specified in Chapter 3, and the testing performed to verify that the core functionality **F1** (per-flow classification of submitted traffic) behaves as required. The chapter is organised top-down: it starts from the architectural breakdown, drills into each component's design, walks through the implementation choices that turn the design into running code, and finishes with the testing strategy and an executable user manual for **F1**.

4.1 Architectural Overview

The system is split into a training subsystem (offline) and a serving subsystem (online), as introduced in Section 3.6. The two are decoupled by a small, well-defined contract: a fixed number of serialised artefacts per trained model variant. This decoupling is deliberate. It lets the training side iterate freely (different splits, granularities, hyperparameters) without forcing changes to the serving code, and it lets the serving code stay tiny and observable.

4.1.1 Component Decomposition

Table 4.1 lists the components of the application together with the responsibility of each. Two design intents drive the decomposition. First, every component has a single concern, so a fault is locatable by pointing at the responsibility table. Second, the boundary between components is the artefact contract, not a network protocol or in-process API: this means the training side can be re-run independently of any deployed serving instance, and conversely the serving side can be reloaded by simply pointing it at a different artefact directory.

Component	Responsibility
Training pipeline	Ingestion, deduplication, per-class subsampling, splitting, scaler/encoder fitting, model training, evaluation, latency benchmarking, artefact persistence.
Inference component	Loads the scaler, encoder, and model weights for a given (split, granularity) variant; preprocesses incoming feature rows; runs the model; returns labelled predictions and confidences.
Capture-to-CSV bridge	Locates a CICFlowMeter backend, runs it as a subprocess against a packet capture, and returns the path of the produced feature CSV.
Web application	Discovers available model variants on disk; exposes the user interface; routes upload requests through the inference component; formats and returns verdicts.
Project specification	Project-level invariants (feature set, split strategies, classification granularities, modelling constraints) that bind every component above.

Table 4.1: Components of the application and the responsibility of each.

4.1.2 Artefact Contract

The serving subsystem consumes exactly three artefacts per model variant. They are produced by the training pipeline and read back by the inference component:

- the fitted feature scaler for the split (one scaler per split, shared across granularities, because the input feature space is identical regardless of label encoding);
- the fitted label encoder for the (split, granularity) pair (one per granularity, because the label set changes);
- the trained model weights for the (split, granularity) pair.

The artefact filenames encode both the split and the granularity in a uniform pattern. As a consequence, the web application can probe the artefact directory at startup, parse each filename, and instantiate one inference component per complete artefact triple it finds — with no hard-coded list of variants. Exposing a newly trained variant in the user interface is therefore purely a question of dropping its three files into the artefact directory and restarting the application.

4.2 Data Pipeline Design

The training data pipeline performs a sequence of deterministic transformations from the raw per-folder CSVs to the train/validation/test tensors that feed the model. Each transformation is a separate pipeline phase so that intermediate state is inspectable.

4.2.1 Ingestion and Deduplication

For each of the attack folders, the pipeline lazily scans every CSV, projects only the feature columns plus a synthetic column that carries the source-capture filename, and concatenates the resulting lazy frames. Materialisation happens only after the projection, so the multi-tens-of-millions-of-rows corpus is never held in memory in its raw width.

CICIoT2023 contains a substantial fraction of exact duplicate rows — a known property of the dataset and a recurring concern in the literature. Deduplication is applied immediately after concatenation, before any sampling. Deduplicating before sampling is essential: a folder where most rows are duplicated would otherwise have the same row repeated many times in the sample, which would inflate accuracy on the corresponding test rows.

4.2.2 Per-Class Subsampling

After deduplication, every folder whose deduplicated row count exceeds the per-class cap is randomly subsampled to that cap with a fixed seed. This brings the effective training-set size down from tens of millions of rows to a few million while keeping every attack class represented.

The cap is set at the high end of the range used in the CICIoT2023 literature. Random sampling is preferred over taking the leading rows of each folder because each attack folder is a chronological concatenation of several capture sessions: the first N rows come almost entirely from the first session, and a leading-rows sample would bias the model toward whatever happened in that opening session.

4.2.3 Caching

The deduplicated, subsampled, labelled rows are concatenated across folders and written to a single Zstandard-compressed columnar file. Subsequent runs skip ingestion and load from this cache directly. Regeneration is forced by deleting the cache file — this is documented inline so future runs do not silently use a stale cache after sampling logic changes.

4.2.4 Cleaning

The cleaning phase performs three operations on the materialised frame. First, null-label rows are dropped (defensive only — labels come from folder names, so this should be a no-op). Second, infinities are replaced with nulls; the actual median imputation is deferred to after the train/validation/test split so the medians are computed on the training partition only. Third, a $\log(1 + x)$ transform is applied to the heavy-tailed continuous features (rates, total bytes, inter-arrival times). Binary protocol and flag indicators are excluded from the log transform.

4.2.5 Three Split Strategies

Three splitting functions are defined and selected by configuration. Their design follows directly from the requirements in Section 3.3.1.

Random row split. Two stratified splits in series produce 70/15/15 train/validation/test partitions, stratified by the fine-grained label so every class appears in every partition. This is reported only for parity with previously published numbers.

Per-capture split. Two consecutive group-aware splits with the source-capture filename as the group identifier produce a 70/15/15 split where no source capture ever appears in more than one partition. This eliminates within-session redundancy leaking across the boundary, but preserves no temporal order.

Temporal split (headline). For each attack folder, the source captures are sorted by filename and partitioned chronologically: the earliest 70% become train, the next 15% become validation, and the latest 15% become test. Folders with fewer than three distinct source captures fall back to a row-level temporal split inside the folder. This mirrors deployment — train on past, test on future — and is the headline result reported in the thesis. The temporal F1 is expected to sit a few points below the random F1; that gap is the honest generalisation story.

4.2.6 Train-Only Preprocessing Fit

For each split, column medians are computed on the training rows only, nulls in the train, validation, and test partitions are imputed using those medians, and the robust scaler is fitted on training rows alone before being applied to all three partitions. The contract that medians and the scaler are fitted strictly on training rows enforces NFR-3 (no test-set leakage) and is what makes the temporal-split metrics meaningful.

4.3 Model Architecture and Training

4.3.1 MLP Topology

The model is a two-hidden-layer multi-layer perceptron with batch normalisation and dropout:

$$n_{\text{features}} \xrightarrow{\text{Linear}} 128 \xrightarrow{\text{BN, ReLU, Dropout}_{0.3}} 64 \xrightarrow{\text{BN, ReLU, Dropout}_{0.3}} n_{\text{classes}}$$

The architecture is intentionally small. Three considerations drive this choice. First, every empirical study of MLPs on tabular data has reported diminishing returns past two hidden layers when the input dimensionality is in the tens. Second, batch normalisation is required to stabilise training under large class-weight swings: at the fine-grained granularity, the heaviest class weight is more than fifty times the lightest, and gradients without batch normalisation would oscillate. Third, the 0.3 dropout rate is a deliberately conservative regularisation choice that empirically generalises better than the more common 0.5 on this dataset, where the cleaned training set is already several million rows.

4.3.2 Class-Weighted Cross-Entropy

Class weights are computed using the standard balanced-frequency rule applied to the encoded training labels, then passed to the cross-entropy loss. The justification is given in Section 3.3.1: SMOTE would generate synthetic flow vectors that have no operational meaning (a half-and-half SYN-flood / DNS-spoofing flow does not correspond to any real attack), and weighted batch sampling would be redundant with weighted cross-entropy.

4.3.3 Optimiser, Schedule, and Early Stopping

Training uses Adam with an initial learning rate of 10^{-3} , paired with a learning-rate schedule that halves the rate after two consecutive epochs without validation-loss improvement. A separate early-stopping mechanism with a patience of five epochs caps total training. The batch size is 4096, large enough to amortise the batch-normalisation overhead and small enough to keep the per-step gradient meaningful.

The training loop tracks training loss, validation loss, and the current learning rate per epoch, restoring the best validation-loss state at the end. The best state is what is serialised; the final-epoch state is discarded.

4.4 Inference Component Design

The inference component is the single seam between trained artefacts and live requests. Its constructor takes the artefact directory, the split name, and the granularity, and loads the matching triple. Its prediction method takes a frame of feature rows and returns the predicted labels, the per-prediction confidences, the full softmax probabilities, and the class-name list in the encoder's index order.

4.4.1 Preprocessing Mirror

The trickiest part of the inference component is matching the training-time preprocessing exactly. Three rules are enforced inside its preprocessing path, each justified by a corresponding training-time decision:

1. The expected feature columns are validated and reordered, so that column ordering does not silently shift between the trained scaler and the live data.
2. Infinities are replaced with nulls, then the $\log(1 + x)$ transform is applied to the continuous features only. The flag and protocol indicators are kept binary.
3. Residual nulls are filled with zeros after the transform. This is a defensive fill: at training time, median imputation handled nulls, but the trained scaler operates in the transformed space, so a residual null in a live request is filled with a constant rather than a re-derived median.

The transformed array is then passed through the scaler's transform method — never a re-fit. The scaler is treated as immutable at inference time.

4.4.2 Confidence and Label Decoding

The model emits logits; a softmax along the class dimension produces per-class probabilities; the argmax selects the predicted class index; the encoder's inverse transform maps the index back to the human-readable label. The maximum softmax probability is reported as confidence. Confidence here is not a calibrated probability; it is reported because end users (committee, presenter) intuitively expect a number alongside each verdict, and the maximum softmax is the cheapest reasonable proxy.

4.5 Capture-to-CSV Bridge Design

The bridge component exists for one reason: the training data was derived from an established flow extractor, and the project must not reimplement flow extraction in

Python on pain of subtle numerical drift between live and training distributions. The component wraps two backends and returns the path of the produced feature CSV.

4.5.1 Backend Discovery

The original Java CICFlowMeter is preferred. It is detected via an environment variable that points at a built CICFlowMeter installation, and is invoked through the launcher script appropriate to the host platform (one variant for Linux/macOS, one for Windows). If the Java backend is not present, the bridge falls back to a community Python port located through the executable search path. If neither backend is available, an error is raised carrying a configuration message that names both options. The two backends differ in output filename convention; the bridge renames the produced file to a single canonical form so that downstream code does not need to branch on backend.

4.5.2 Subprocess Hygiene

Both backends are launched with their standard output and standard error captured. A non-zero exit from either backend is translated into a domain-level error carrying the captured standard error. This is what surfaces in the user interface under FR-D4: the user sees the underlying tool's diagnostic message, not a Python traceback.

4.6 Web Application Design

The user interface is a thin orchestration layer over the components above. Three design choices are worth calling out.

4.6.1 Filesystem-Driven Variant Discovery

At startup, the application probes the artefact directory for trained model variants. For each filename matching the artefact-naming convention, it parses the encoded split and granularity, attempts to load the corresponding triple, and silently skips incomplete triples. If no triples can be loaded, the application fails fast with a message pointing the user back at the training pipeline. The result is a dictionary keyed by a human-readable display string that becomes the contents of the variant dropdown.

4.6.2 Two Symmetric Upload Paths

The interface exposes two tabs: one for pre-extracted CSV uploads and one for raw packet captures. They share the same dropdown, the same predictor pool, and the

same output schema (summary line plus verdict table); they differ only in the input handler. The CSV tab delegates straight to the inference component; the packet-capture tab creates a temporary directory, calls the bridge to produce a CSV, and feeds that CSV through the same classification path used by the CSV tab. This shared post-extraction code path means the verdict display behaves identically regardless of how the CSV was obtained, which simplifies both the user mental model and the testing surface.

4.6.3 Localhost Binding by Default

The application binds to all local interfaces by default and is reached in a browser at the localhost URL. Public exposure is delegated to a separate command-line tunnelling tool, started in another terminal when needed. This separation is intentional: tunnelling is a presentation-time decision and should not be coupled to the application's lifecycle.

4.7 Implementation Notes

The previous sections covered *why* each component looks the way it does. This section walks through the implementation choices made along the way.

4.7.1 Memory and I/O Discipline

The pipeline is built around three rules that together respect the 16 GB memory budget (NFR-4):

- Lazy frames are used everywhere ingestion touches raw CSVs. Materialisation is deferred until after column projection.
- The deduplicated and subsampled frame is the only materialised object on the data side. Once it has been converted to numpy arrays for the split-and-train path, the frame's memory is reclaimed before the training tensors are allocated.
- The cached intermediate is the only persistent caching boundary. There is no in-memory caching that survives across pipeline restarts.

4.7.2 Determinism

A single deterministic seed is broadcast to every random source: the numerical library, the deep-learning framework on CPU and on GPU, the framework's deterministic backend flags, and the dataset samplers. Both random-row split calls receive the

same seed; both group-aware split steps receive the same seed; the temporal split is deterministic by construction. As a result, re-running the pipeline against the same source files produces metrics that match within a few thousandths of an F1 point — the tolerance committed in NFR-3.

4.7.3 Logging and Observability

The pipeline prints concise progress lines at every phase: per-folder ingestion counts (raw → deduplicated → sampled), per-split row counts, missing-class warnings, per-epoch training and validation loss, and final per-class metrics. Plots — training curves and row-normalised confusion matrices — are rendered inline. The web application prints only its bind address on startup and any error message that surfaces during a request.

4.7.4 Configuration Surface

The pipeline exposes its configuration through a small block of named constants near the top of the notebook: the per-class row cap, the active split strategies, the active classification granularities, the batch size, the maximum number of epochs, the early-stopping patience, the learning rate, and the deterministic seed. These are the only knobs intended to be tuned. An iteration mode (one split, one granularity) and a full thesis-run mode (three splits, three granularities) are documented at the configuration block so iteration cost is predictable.

4.7.5 Dependency Footprint

The runtime dependencies are confined to the standard scientific-Python stack (a lazy columnar engine, a deep-learning framework, a tabular machine-learning library, plotting libraries) plus a lightweight web-UI framework and a gradient-boosted-trees implementation for the tree baselines. There is no machine-learning serving framework: the demo loads the model directly into the application process. There is no packaging beyond a flat directory; a single command launches the demo, satisfying NFR-7.

4.8 Testing Strategy

The application is tested at three levels: shape-and-determinism sanity checks inside the training pipeline; integration smoke tests over each (split, granularity) artefact triple; and the F1 functional acceptance test, which is the executable demonstration required by the assignment.

4.8.1 Pipeline-Level Sanity Checks

The training pipeline contains assertions and printed sanity checks at every transition. The most important ones are:

- An assertion that the fine-grained label mapping covers every attack folder exactly once — guards against a missing or duplicated entry.
- An assertion during ingestion that no folder is unmapped — guards against a new attack folder being silently dropped.
- For each split, a check that every fine-grained label appears at least once in the train partition; missing classes are flagged with a warning line.
- Per-folder row counts (raw / deduplicated / sampled) are printed during ingestion — a sudden mismatch between raw and deduplicated counts is a quick visual cue that something has changed in the source data.

4.8.2 Artefact Round-Trip Test

For every (split, granularity) pair, after training, the pipeline reloads the freshly written artefacts using the exact code path the demo uses and predicts on the test partition. The reloaded predictions are compared against the in-memory predictions of the still-loaded model. The two must agree exactly. This test caught two real bugs during development: a column-ordering drift between training and inference, and a silent dtype mismatch when arrays were converted to tensors without an explicit precision cast.

4.8.3 Latency Benchmark

For each trained model, the benchmark loop runs 10,000 inference passes per batch size in $\{1, 32, 256, 1024\}$ on CPU after a warmup of one hundred passes, with the timer wrapped around the feature scaling step *and* the model forward pass. The pipeline reports the p50, p95, and p99 of the per-batch latency in milliseconds and the implied flows-per-second throughput. NFR-1 (sub-100 ms p95 end-to-end on a consumer laptop CPU at any of those batch sizes) is the acceptance criterion.

4.8.4 F1 Functional Test — Capture to Verdict

F1 is verified end-to-end with two test inputs whose ground-truth labels are known, so the test is binary pass/fail rather than dependent on subjective inspection. Each test consists of running the demo, uploading the input, and reading the predicted-label distribution.

- **Benign baseline.** A short capture of normal browsing traffic produced on the demo machine. Acceptance criterion: at the binary granularity, at least 95% of flows are predicted as Benign.
- **Synthetic flood.** A capture of a SYN-flood attack directed at a local target, generated using a standard packet-crafting tool. Acceptance criteria: at the binary granularity, at least 95% of flows are predicted as Attack; at the attack-family granularity, the dominant predicted family is DDoS.

These thresholds are deliberately set well below the test-set numbers (which are typically 0.97+ in the binary case) so the test tolerates the inevitable distribution shift between the training data and the live demo machine, while still failing loudly if the model has been deployed against a wrong scaler or a wrong encoder.

4.8.5 Negative-Path Testing

Three negative scenarios are exercised manually. They rely on the public component surface, so they can be reproduced from a brief interactive session:

- Uploading a CSV missing one or more of the expected feature columns. Expected: a domain-level error naming the missing columns; the user interface surfaces this as the summary message and produces an empty table. Verifies FR-D4.
- Uploading a non-capture file (e.g. a text file renamed to a capture extension). Expected: a flow-extraction error carrying the underlying tool's stderr; the user interface surfaces this as the summary message. Verifies FR-D4.
- Running the demo with no artefacts on disk. Expected: a fast exit with a message pointing the user back to the training pipeline. Verifies graceful behaviour when discovery yields nothing (FR-D2).

4.9 F1 User Manual

This section is the executable mini user manual for F1, suitable for inclusion in the thesis defense package. It assumes the artefacts have already been trained (see UC-1 in Section 3.5.1) and that the working directory is the project root.

4.9.1 Step 1 — Install Dependencies

Activate the project's virtual environment and install the runtime dependencies:

```
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Linux/macOS
pip install -r requirements.txt
```

4.9.2 Step 2 — Provide a CICFlowMeter Backend

For packet-capture input, one of two backends must be reachable. Either set the environment variable that points at a built CICFlowMeter installation:

```
$env:CICFLOWMETER_HOME = "C:\path\to\CICFlowMeter" # PowerShell
```

or install the Python community port:

```
pip install cicflowmeter
```

CSV-only operation (UC-3) does not require either backend.

4.9.3 Step 3 — Launch the Demo

Run the application as a module from the project root:

```
python -m demo.app
```

The application prints its bind address. The interface is reached in a browser at `http://localhost:7860`. (The all-interfaces bind address shown in the terminal is not a connection address and using it directly in the browser fails on Windows.)

4.9.4 Step 4 — Choose a Model Variant

The dropdown labelled “Model (split / granularity)” lists every artefact triple discovered on disk. The default selection is the first variant returned by the discovery probe; for the thesis demonstration the recommended variant is the temporal split at the binary granularity, which is the headline combination defended in the methodology.

4.9.5 Step 5 — Submit a Capture

Two upload paths are exposed:

- **CSV upload.** Switch to the CSV upload tab. Click the file picker, select a CIC-format CSV, then click the Classify button. The summary line and verdict table appear within a second on a typical laptop.

- **Packet-capture upload.** Switch to the packet-capture upload tab. Select a .pcap or .pcapng file. Click the Extract-and-Classify button. The capture is processed by the flow-extraction tool (typically a few seconds for short captures); the verdict table appears immediately afterwards.

4.9.6 Step 6 — Read the Output

Two artefacts are returned for every classification request:

- A **summary** string of the form “ N flows classified — LabelA: n_A — LabelB: n_B — ...”, sorted by descending count.
- A **verdict table** with three columns: row index in the input, predicted label, and the maximum-softmax confidence formatted with four decimal digits.

4.9.7 Step 7 — (Optional) Expose the Demo Publicly

For a remote audience, open a second terminal alongside the running demo and start an HTTPS tunnel pointed at the local demo URL:

```
cloudflared tunnel --url http://localhost:7860
```

The tunnelling service prints a temporary HTTPS URL. Sharing this URL gives a remote viewer access to the demo. Stopping the tunnelling tool revokes the URL.

4.9.8 Step 8 — Capture Screenshots for the Defense Package

For the assignment deliverable, record the executable evidence of F1 by capturing three screenshots:

1. The demo launching in the terminal (showing the bind address it prints).
2. The user interface immediately after a benign capture has been classified — the summary line should read primarily Benign.
3. The user interface immediately after a synthetic-flood capture has been classified — the summary line should read primarily Attack at the binary granularity, or primarily DDoS at the attack-family granularity.

Figures 4.1–4.3 show the layout placeholders for these three screenshots; the actual images are produced during the live demonstration and inserted into the defense package.

```
(.venv) PS G:\uni\cic-iot2023-detection-system> python -m demo.app

* Running on local URL: http://0.0.0.0:7860
* To create a public link, set `share=True` in `launch()`.
█
```

Figure 4.1: Demo launch in the terminal — the application boots, discovers the trained model artefacts, and binds the user-interface URL.

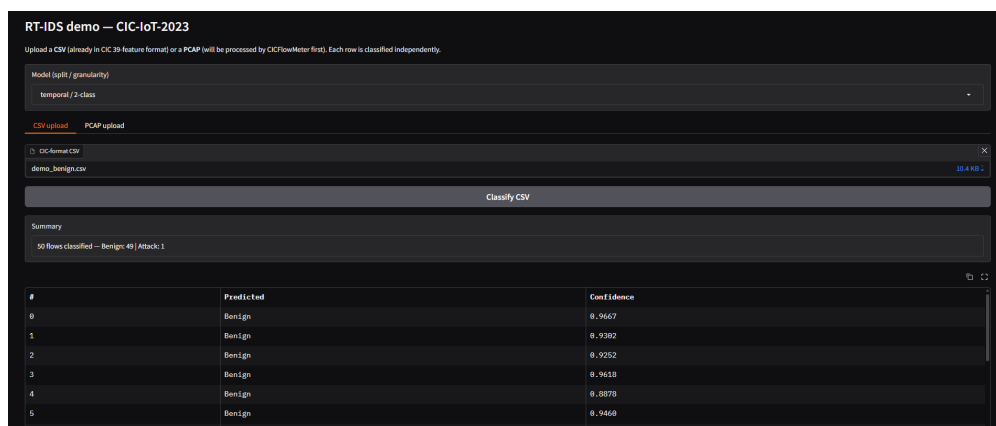


Figure 4.2: F1 acceptance — benign capture classified. The summary line and verdict table report Benign as the dominant predicted label, satisfying the no-false-alarm half of the F1 acceptance criterion (Section 4.8.4).

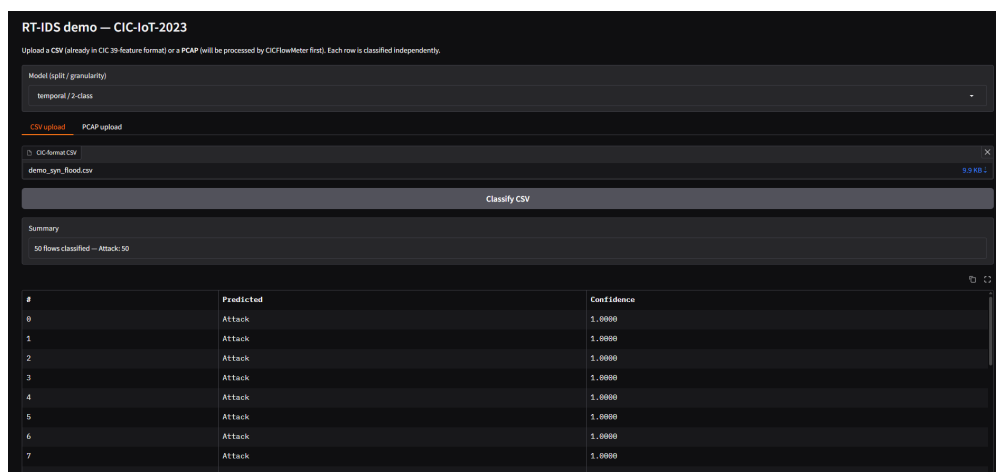


Figure 4.3: F1 acceptance — synthetic SYN-flood capture classified. The summary line and verdict table report Attack as the dominant predicted label, satisfying the discrimination half of the F1 acceptance criterion (Section 4.8.4).

Chapter 5

Results and Discussion

This chapter will report experimental results across the three split strategies and the three classification granularities, the comparison against tree-based baselines (Random Forest, XGBoost), the latency benchmark, and a discussion of per-attack-family detection performance — with particular attention to the expected weaker detection of web-family attacks. (To be completed in a subsequent assignment.)

Chapter 6

Conclusions and Future Work

This chapter will summarize the findings of the thesis and outline directions for future work. (To be completed in a subsequent assignment.)

Bibliography

- [1] Hassan M. J. Almohri et al. Explainable artificial intelligence for intrusion detection in iot networks: A deep learning based approach. *Expert Systems with Applications*, 238:121855, 2024.
- [2] Moayad Aloqaily, Safa Otoum, Ismaeel Al Ridhawi, and Yaser Jararweh. Aids: Application of deep learning to realtime web intrusion detection. In *2020 International Wireless Communications and Mobile Computing (IWCMC)*, pages 2058–2063. IEEE, 2020.
- [3] Giuseppina Andresini, Feargus Pendlebury, Fabio Pierazzi, Corrado Loglisci, Annalisa Appice, and Lorenzo Cavallaro. INSOMNIA: Towards concept-drift robustness in network intrusion detection. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security (AISec '21)*, pages 111–122. ACM, 2021.
- [4] Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. SoK: Pragmatic assessment of machine learning for network intrusion detection. In *Proceedings of the 8th IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 592–614, Delft, Netherlands, 2023. IEEE.
- [5] Mohamed Amine Ferrag, Leandros Maglaras, Sotiris Moschoyiannis, and Helge Janicke. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50:102419, 2020.
- [6] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 23928–23941, 2021.
- [7] Jiyeon Kim, Jiwon Kim, Hyunjung Kim, Minsun Shim, and Eunjung Choi. CNN-based network intrusion detection against denial-of-service attacks. *Electronics*, 9(6):916, 2020.
- [8] Jan Lansky, Saqib Ali, Mokhtar Mohammadi, Mohammed Kamal Majeed, Sarkhel H. Taher Karim, Shima Rashidi, Mehdi Hosseinzadeh, and Amir Ma-

- soud Rahmani. Deep learning-based intrusion detection systems: A systematic review. *IEEE Access*, 9:101574–101599, 2021.
- [9] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2018. Internet Society.
- [10] Muhammad Nadeem et al. Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors*, 23(13):5941, 2023.
- [11] Martin Roesch. Snort – lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238, Seattle, Washington, USA, 1999.
- [12] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.
- [13] R. Vinayakumar, Mamoun Alazab, K. P. Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, and Sitalakshmi Venkatraman. Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7:41525–41550, 2019.
- [14] R. Vinayakumar, K. P. Soman, and Prabaharan Poornachandran. Applying convolutional neural network for network intrusion detection. In *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 1222–1228. IEEE, 2017. Frequently cited under year 2018 due to indexing date.
- [15] Zihan Wu, Hong Zhang, Penghai Wang, and Zhibo Sun. RTIDS: A robust transformer-based approach for intrusion detection system. *IEEE Access*, 10:64375–64387, 2022.